
WEATHER-FISH: SYSTEM ARCHITECTURE, AI INTEGRATION, AND DEPLOYMENT

INDIVIDUAL CONTRIBUTION REPORT

Fenil Ramani

Project Lead

Faculty of Electrical Engineering, Media and Computer Science
OTH Amberg-Weiden
`f.ramani@oth-aw.de`

July 11, 2026

ABSTRACT

This report explains, in plain terms, what I personally built for WEATHER-FISH — an AI-driven weather app developed by a three-person team at OTH Amberg-Weiden. The project brief names me as Project Lead, responsible for “*system architecture, AI integration, prompt engineering, and deployment*”, while my teammates Prasad Rajyguru and Shubham Kushwaha handled the frontend/audio and the backend routes/scheduler. This report covers only my part: the Context Engine that decides the tone of a report, the AI model that writes the text, the safety-net fallback that keeps the app working when the AI provider runs out of quota, the chat assistant built on top of it, the worldwide city search, and the deployment to Hugging Face. Diagrams are included to make the design easy to follow. The report also explains how my work connected to my teammates’ work, and one real integration mistake that came up and how I fixed it.

Keywords Individual contribution · System architecture · AI integration · Prompt engineering · Fallback design · Deployment

1 Introduction

WEATHER-FISH turns a plain weather app into one that writes and speaks a personalised weather report, instead of just showing numbers on a screen. Reports are generated ahead of time, so they play back instantly, and their wording changes automatically depending on whether the weather is calm, dangerous, or great for outdoor plans [Ramani, 2026].

The project brief splits this into three jobs (Table 1). My job, as Project Lead, is “*system architecture, AI integration, prompt engineering, deployment*” [Ramani, 2026]. In the system flow diagram from the brief — *Weather API* → *Context Engine* → *Prompt Engineering* → *AI Model* → *TTS* → *Storage* → *Playback* — that means I built the three middle stages, plus the overall design that connects every stage, plus getting the finished app online.

This report only covers my own work and how it connects to the rest of the team. It does not describe my teammates’ code in detail.

2 Who Did What

Table 1: Team responsibilities, exactly as written in the project brief [Ramani, 2026]. Only my row is explained further in this report.

Team member	Job
Fenil Ramani (me), Project Lead	System architecture, AI integration, prompt engineering, deployment.
Prasad Rajyguru	Frontend, UI/UX, audio playback, user interaction.
Shubham Kushwaha	Backend, API integration, scheduler, storage.

Figure 1 shows the architecture as a cycle rather than a stack, because that is what it actually is: the same six steps repeat automatically, once an hour, without anyone touching the app. Gold steps are mine; grey steps were built by the rest of the team (Table 1); the whole cycle runs inside the single container I deployed.

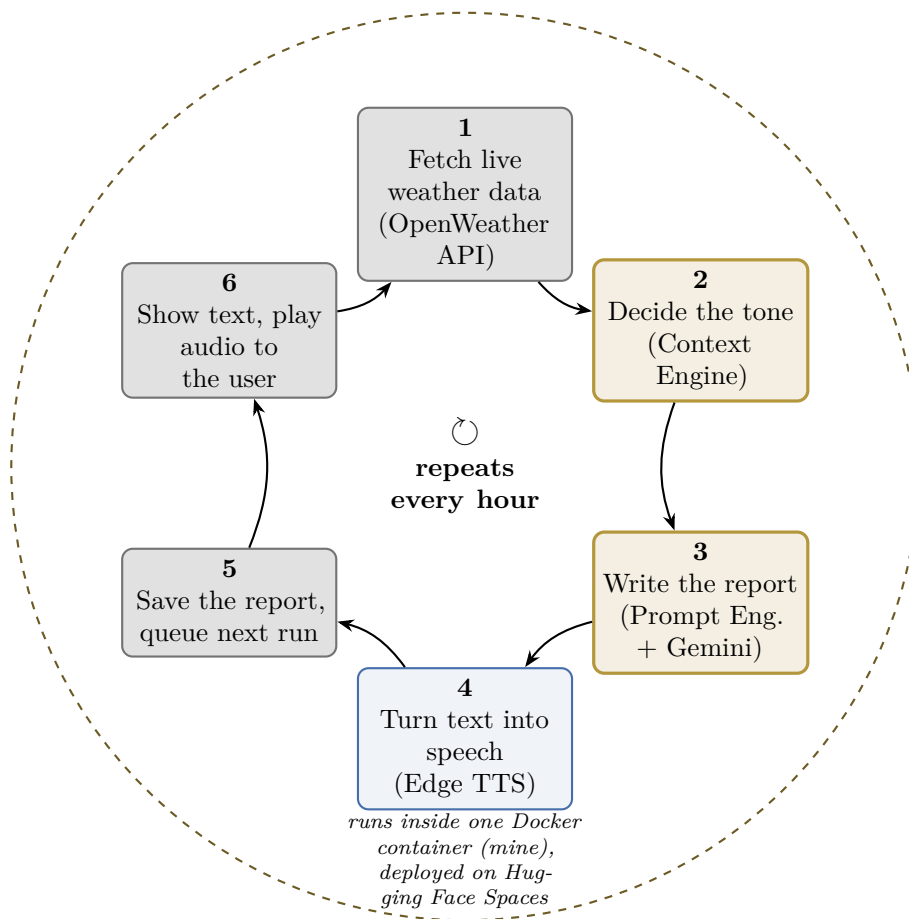


Figure 1: The WEATHER-FISH cycle: one automated loop, triggered every hour by the scheduler. Gold = my steps; grey = built by the rest of the team; the dashed ring is the single Docker container the whole loop runs inside.

3 What I Built

3.1 Context Engine: deciding the tone

The brief says the Context Engine must look at weather severity, time of day, and user preferences, and turn that into simple labels such as *warning*, *advisory*, or *cheerful* [Ramani, 2026]. I built

this as a small, rule-based Python function. It does not use AI — it uses plain if/else logic on temperature and rain data. I made this choice on purpose: a storm warning should always be described as urgent, and a rule always does that the same way, whereas an AI model might phrase it differently each time.

```

1 {
2   "severity": "warning", "time_of_day": "morning",
3   "tone": "cautionary and urgent -- warn the user clearly",
4   "highlights": ["current rainfall -- recommend an umbrella"],
5 }

```

Listing 1: What the Context Engine outputs. This small object is then handed to the AI model as instructions.

3.2 AI Model and Fallback: writing the report

This is the core AI-integration task from the brief. I connect to Google Gemini and ask it to write the report text, using the Context Engine’s labels to set the tone (this is the “prompt engineering” part). The one thing I had to design around is quota: a free Gemini account can only be used a limited number of times per day. If I only used one model, the whole app would stop writing new reports once that daily limit was hit.

My fix was to try several models, one after another, and only fall back to a fixed, no-AI template if every single one is out of quota. Figure 2 shows this as a simple flowchart.

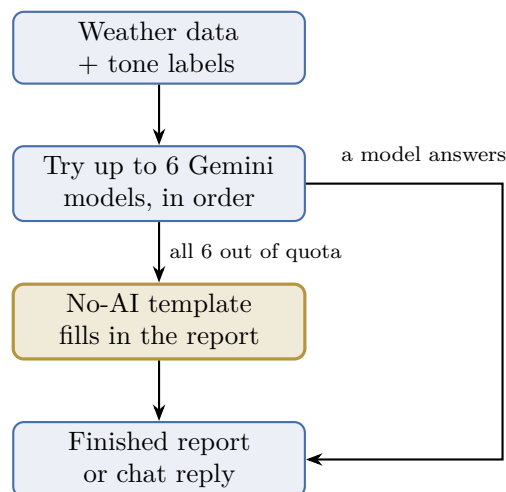


Figure 2: My fallback design: the app always finishes with a report, even if the AI model is completely unavailable. Gold boxes are the safety-net path.

```

1 MODELS = ["gemini-2.5-flash-preview-05-20", "gemini-2.5-flash",
2           "gemini-2.0-flash", "gemini-2.0-flash-lite",
3           "gemini-1.5-flash-001", "gemini-1.5-flash-8b-001"]

```

Listing 2: The six models I use, in the order I try them. Each one has its own separate daily quota.

On top of this, I wrote three presenter personalities (Fisch, Merkel, Haftbefehl) as prompt instructions, so the same weather data can be narrated in three very different voices, plus a small randomised instruction that stops the AI from writing the same opening sentence every time.

3.3 Chat Assistant

I reused the same Context Engine and the same model list to build a chat assistant, so it always agrees with the report the user is already looking at. Its most useful feature does not need AI

at all: if a user asks something like *“good for cycling today?”*, the assistant checks rain chance, temperature, and wind speed directly against simple thresholds and gives a clear yes/no answer. This means activity questions still work even on a day when the AI quota is completely used up, in both German and English.

3.4 Worldwide City Search

The app originally only understood German postal codes. I changed this so any city in the world can be searched: the frontend sends a search text to my backend, my backend asks a geocoding service for matching cities, and returns them as a simple list. I picked an identifier format (latitude_longitude, e.g. "22.3000_70.8000") that did not require changing how reports are already stored, so this upgrade did not break anything my teammates had already built on the old German-only format.

3.5 Deployment

The last part of my job is deployment. I packaged the whole app — backend, AI layer, and the built frontend — into one Docker container and put it on Hugging Face Spaces, so anyone can open it in a browser without installing anything, matching the brief’s goal.

4 Working with My Teammates

Because my three AI stages sit between the other two teammates’ stages, I made sure every one of my endpoints returned a fixed, predictable shape of data, and treated that shape as a promise to the rest of the team. For example, my city-search endpoint always returns a list of {name, country, lat, lon, id} objects, and my report endpoint always accepts {cities, zipcodes, language, hobbies}. This let my teammates build their side without waiting for me to finish, or knowing how my AI code works internally. Figure 3 shows the two places this handoff happens in practice.

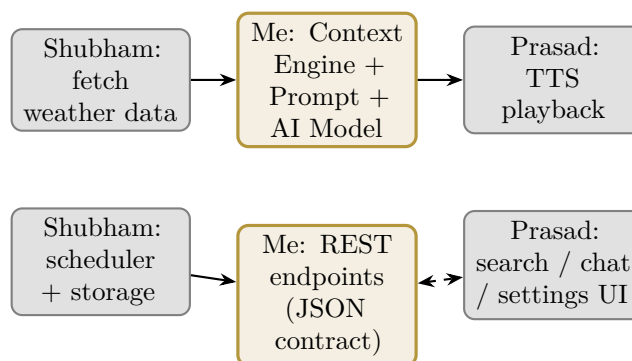


Figure 3: How the three of us worked together. Top row: the data pipeline handoff (Shubham’s weather data → my AI layer → Prasad’s audio playback). Bottom row: the two-way interface contract between my REST endpoints and the scheduler/frontend code that calls them — this is exactly where the Settings-page bug described below happened.

That said, one mistake still slipped through. When I switched location IDs to the new worldwide format, I updated the backend and one of the two search boxes in the frontend — but a second, older search box (on the Settings page) still only searched the old German-only list, and nobody had noticed. Users typing a non-German city there simply saw no results, even though the backend fully supported it. I only found this by testing the live page myself, not by reviewing the backend code alone, because more than one search box existed after several rounds of changes. Fixing it meant checking exactly which search box the app actually used. Since then, I treat a change like this as “done” only after I have checked every place that uses it — not just the one I was thinking about when I made the change.

We also deployed a new working version every couple of weeks instead of holding changes until the end. This meant problems like the one above showed up quickly, instead of piling up until a final integration week.

5 Results

Table 2: Outcomes from the work described in Section 3.

What I built	Result
Six-model AI fallback	About $6\times$ more reports per day than using one AI model, for the same free quota.
No-AI template fallback	The app never shows an empty report, even when AI quota runs out completely.
Rule-based activity advice	Correct yes/no activity answers with zero AI calls, in German and English.
Worldwide city search	Any searchable city works now, not just German ZIP codes — with no change needed to how reports are stored.
Deployment	The whole app runs from one container on Hugging Face Spaces.

6 What I Learned

Building a safety net for the AI model mattered more than any single feature, because the free daily quota ran out often during testing — it was a normal, everyday problem, not a rare edge case. I also learned that a design change is not really finished once my own code works: the Settings-page mistake in Section 4 happened because I checked my own change but not every place in the app that depended on it. I now double-check every user of a changed interface, not just the one I had in mind.

7 Declaration of AI Tool Usage

Tools used: Claude (Anthropic), used as a coding assistant inside my editor.

Purpose:

- Debugging help during backend development, e.g. tracing a type-checking error in the chat module, and diagnosing the Settings-page search bug described in Section 4.
- Turning my own notes on what I built into clear, structured writing for this report, which I then reviewed and edited myself.
- *Google Gemini* (part of the app I built, not a writing tool) is the AI model that writes the reports and chat replies described in Section 3.

I reviewed and tested every AI-assisted suggestion before using it. The ideas, analysis, and conclusions in this report are my own.

References

- Ramani, F. (2026). *WEATHER-FISH — Project Definition*. OTH Amberg-Weiden, internal project brief, April 2026.
- Google LLC (2024). *Gemini API Documentation*. <https://ai.google.dev/gemini-api/docs>.
- Tiangolo, S. (2023). *FastAPI Documentation*. <https://fastapi.tiangolo.com>.
- OpenWeather Ltd (2024). *OpenWeather One Call API 3.0 and Geocoding API Documentation*. <https://openweathermap.org/api/one-call-3>.